

Struttura del Sistema

Microservizi

Il sistema possiede i seguenti microservizi:

- **API Gateway:** è il punto di accesso centrale. Riceve le richieste e le invia al microservizio giusto.
- **Auth Service:** gestisce l'autenticazione e l'autorizzazione. Utilizza AWS Cognito e gestisce la JWT.
- **Catalog Command Service:** gestisce le operazioni di modifica del catalogo, come la pubblicazione di brani e la creazione di album. Si occupa anche del caricamento dei file audio su AWS S3 e invia eventi quando il catalogo viene modificato.
- **Catalog Query Service:** gestisce le richieste di lettura del catalogo. Riceve gli eventi dal Command Service e aggiorna il proprio database. Utilizza Redis per velocizzare le richieste più frequenti.
- **Engagement Service:** gestisce i dati degli utenti, come i like, gli ascolti e le playlist. Contiene anche i processi eseguiti in background, ad esempio l'invio di email automatiche.

Comunicazione tra i servizi

I microservizi comunicano tra loro in due modi:

- **Comunicazione sincrona:** viene usato **RestClient** quando un servizio ha bisogno di ottenere dei dati subito da un altro microservizio.
- **Comunicazione asincrona:**
 - **Code Messaggi:** Servono per mettere in coda operazioni specifiche. Garantisce che il messaggio venga elaborato almeno una volta (at-least-once).
 - **Publisher/Subscriber:** Usato nel pattern CQRS. Il Catalog Command Service pubblica eventi, come "Canzone Pubblicata". Il Catalog Query Service li riceve e aggiorna il database di lettura, mantenendolo sincronizzato nel tempo (consistenza eventuale).

Ambiente Docker

Tutti i servizi del sistema (microservizi di Spring, database, broker dei messaggi etc.) vengono lanciati tramite container Docker.

Un **docker-compose.yml** permette di avviare tutti i servizi.